

Problem 1: Normalize it! Going from ONF to 3NF

The following is a very denormalized dataset representing information about customers, their orders, and the products they purchase. The goal is to normalize this data from ONF to 3NF, providing explanations for each step:

Step 0: Understand ONF

Customer_ID	Customer_Name	Customer_Phone	Order_ID	Order_Date	Products	Prices	Shipping_City	Shipping_Zip	Shipping_Country
1	Alice Smith	555-1234	1001	2024-12-01	Laptop, Mouse	1000, 25	Springfield	62704	USA
2	Bob Johnson	555-5678	1002	2024-12-02	Phone, Headphones	800, 50	Springfield	62704	USA

Step 1: Convert to 1NF

Step 2: Convert to 2NF

Step 3: Convert to 3NF

Problem 2: Star Schema

While the data in **3NF** is optimized for transactional systems (OLTP), it may not be efficient for analytical queries, which often require aggregating data across multiple tables. For the analytical layer, we need to transform the normalized schema into a **star schema** or other suitable structure.

1. Design a star schema for the data.
2. Identify the fact table and dimension tables.

Problem 3: Business Analytics

Using the STAR schema you created, write some SQL to answer the following questions:

Q1: What is the total revenue generated by each product?

Q4: What is the revenue trend by month and year?

Q2: What is the total revenue by customer?

Q5: Which customer purchased the most expensive product?

Q3: What is the revenue generated per shipping city?

Problem 4: DatelistInt

Revisit the example from class:

userId	firstActive	LastActive	activeDateList	date
1	2022-01-01	2022-01-05	[2022-01-01, 2022-01-03,2022-01-05]	2022-01-05
2	2022-01-01	2022-01-04	[2022-01-02, 2022-01-04]	2022-01-05

Find the ?:

userId	firstActive	LastActive	activeDateList	date
1	2022-01-01	2022-01-05	21	2022-01-05
2	2022-01-01	2022-01-04	?	2022-01-05

Solutions

Problem 1

Step 1: Convert to 1NF

We remove repeating groups (like multiple products per order) by creating a separate row for each product in an order.

Customer_ID	Customer_Name	Customer_Phone	Order_ID	Order_Date	Product	Product_Price	Shipping_City	Shipping_Zip	Shipping_Country
1	Alice Smith	555-1234	1001	2024-12-01	Laptop	1000	Springfield	62704	USA
1	Alice Smith	555-1234	1001	2024-12-01	Mouse	25	Springfield	62704	USA
2	Bob Johnson	555-5678	1002	2024-12-02	Phone	800	Springfield	62704	USA
2	Bob Johnson	555-5678	1002	2024-12-02	Headphones	50	Springfield	62704	USA

Step 2: Convert to 2NF

We identify functional dependencies and remove **partial dependencies** (attributes that depend only on part of the composite key).

Functional Dependencies:

- **Customer_ID** → **Customer_Name**, **Customer_Phone**
- **Order_ID** → **Order_Date**, **Shipping_City**, **Shipping_Zip**, **Shipping_Country**
- **Order_ID + Product** → **Product_Price**

This results in the following tables:

Customers Table

Customer_ID	Customer_Name	Customer_Phone
1	Alice Smith	555-1234
2	Bob Johnson	555-5678

Orders Table

Order_ID	Order_Date	Customer_ID	Shipping_City	Shipping_Zip	Shipping_Country
1001	2024-12-01	1	Springfield	62704	USA
1002	2024-12-02	2	Springfield	62704	USA
1003	2024-12-05	1	Springfield	62704	USA

Order_Details Table

Order_ID	Product	Product_Price
1001	Laptop	1000
1001	Mouse	25
1002	Phone	800
1002	Headphones	50
1003	Monitor	200

Explanation of 2NF:

- **Customer_Name** and **Customer_Phone** depend only on **Customer_ID** and are moved to the **Customers Table**.
- Shipping information and **Order_Date** depend only on **Order_ID** and remain in the **Orders Table**.
- Each product's price depends on both **Order_ID** and **Product**, so this data is placed in the **Order_Details Table**.

Step 3: Convert to 3NF

We remove **transitive dependencies**, where non-key attributes depend on other non-key attributes rather than directly on the primary key.

Transitive Dependency

In the **Orders Table**, the attributes **Shipping_City**, **Shipping_Zip**, **Shipping_Country** are dependent on **Shipping_Zip** rather than directly on **Order_ID**.

Resolution: We create a new **Shipping Table** to hold shipping-related attributes. This ensures that **Shipping_City** and **Shipping_Country** are only dependent on **Shipping_Zip**.

Customers Table

Customer_ID	Customer_Name	Customer_Phone
1	Alice Smith	555-1234
2	Bob Johnson	555-5678

Orders Table

Order_ID	Order_Date	Customer_ID	Shipping_Zip
1001	2024-12-01	1	62704
1002	2024-12-02	2	62704
1003	2024-12-05	1	62704

Order_Details Table

Order_ID	Product	Product_Price
1001	Laptop	1000
1001	Mouse	25
1002	Phone	800
1002	Headphones	50
1003	Monitor	200

Shipping Table

Shipping_Zip	Shipping_City	Shipping_Country
62704	Springfield	USA

Explanation of 3NF:

- The transitive dependency **Shipping_Zip → Shipping_City, Shipping_Country** is removed.
- The **Shipping Table** ensures that every attribute is only dependent on the primary key.

Problem 2

Fact Table

Fact Table: Sales_Fact

Order_ID	Product_ID	Customer_ID	Shipping_ID	Date_ID	Product_Price
1001	1	1	1	20241201	1000
1001	2	1	1	20241201	25
1002	3	2	1	20241202	800
1002	4	2	1	20241202	50
1003	5	1	1	20241205	200

Dimension Tables

1. Customers_Dim

Holds information about customers.

Customer_ID	Customer_Name	Customer_Phone
1	Alice Smith	555-1234
2	Bob Johnson	555-5678

2. Products_Dim

Holds details about products.

Product_ID	Product_Name
1	Laptop
2	Mouse
3	Phone
4	Headphones
5	Monitor

3. Shipping_Dim

Contains shipping-related information.

Shipping_ID	Shipping_City	Shipping_Zip	Shipping_Country
1	Springfield	62704	USA

4. Date_Dim

Provides date-related attributes for time-based analysis.

Date_ID	Order_Date	Day	Month	Year	Quarter
20241201	2024-12-01	1	12	2024	Q4
20241202	2024-12-02	2	12	2024	Q4
20241205	2024-12-05	5	12	2024	Q4

Problem 3

Q1: What is the total revenue generated by each product?

```
SELECT
    p.Product_Name,
    SUM(f.Product_Price) AS Total_Revenue
FROM
    Sales_Fact f
JOIN
    Products_Dim p ON f.Product_ID = p.Product_ID
GROUP BY
    p.Product_Name;
```

Product_Name	Total_Revenue
Laptop	1000
Mouse	25
Phone	800
Headphones	50
Monitor	200

Q2: What is the total revenue by customer?

```
SELECT
    c.Customer_Name,
    SUM(f.Product_Price) AS Total_Revenue
FROM
    Sales_Fact f
JOIN
    Customers_Dim c ON f.Customer_ID = c.Customer_ID
GROUP BY
    c.Customer_Name;
```

Customer_Name	Total_Revenue
Alice Smith	1225

Bob Johnson	850
-------------	-----

Q3: What is the revenue generated per shipping city?

```
SELECT
    s.Shipping_City,
    SUM(f.Product_Price) AS Total_Revenue
FROM
    Sales_Fact f
JOIN
    Shipping_Dim s ON f.Shipping_ID = s.Shipping_ID
GROUP BY
    s.Shipping_City;
```

Shipping_City	Total_Revenue
Springfield	2075

Q4: What is the revenue trend by month and year?

```
SELECT
    d.Year,
    d.Month,
    SUM(f.Product_Price) AS Total_Revenue
FROM
    Sales_Fact f
JOIN
    Date_Dim d ON f.Order_Date = d.Order_Date
GROUP BY
    d.Year, d.Month;
```

Year	Month	Total_Revenue
2024	12	2075

Q5: Which customer purchased the most expensive product?

```
SELECT
    c.Customer_Name,
    MAX(f.Product_Price) AS Max_Product_Price
FROM
    Sales_Fact f
JOIN
    Customers_Dim c ON f.Customer_ID = c.Customer_ID
GROUP BY
    c.Customer_Name
ORDER BY
    Max_Product_Price DESC
LIMIT 1;
```

Customer_Name	Max_Product_Price
Alice Smith	1000

Problem 4

Revisit the example from class:

userId	firstActive	LastActive	activeDateList	date
1	2022-01-01	2022-01-05	[2022-01-01, 2022-01-03, 2022-01-05]	2022-01-05
2	2022-01-01	2022-01-04	[2022-01-02, 2022-01-04]	2022-01-05

Find the **?**:

userId	firstActive	LastActive	activeDateList	date
1	2022-01-01	2022-01-05	21	2022-01-05
2	2022-01-01	2022-01-04	?	2022-01-05

Current Date: 2022-01-05. User 2 was active on:

- 2022-01-02 (1 days ago)
- 2022-01-04 (3 day ago)

Bit Representation: Each day is mapped to a bit:

- 2022-01-05 $\rightarrow$ Bit 0 $\rightarrow 0 \rightarrow 0 \cdot 2^0 = 0$
- 2022-01-04 $\rightarrow$ Bit 1 $\rightarrow 1 \rightarrow 1 \cdot 2^1 = 2$
- 2022-01-03 $\rightarrow$ Bit 2 $\rightarrow 0 \rightarrow 0 \cdot 2^2 = 0$
- 2022-01-02 $\rightarrow$ Bit 3 $\rightarrow 1 \rightarrow 1 \cdot 2^3 = 8$
- datelist_int 2 + 8 = 10
- **?** = 10